

Assignment No 5

Write X86/64 ALP to perform multiplication of two 8-bit hexadecimal numbers. Use successive addition method only.

```
%macro scall 4          ;macro to take input and output
```

```
    mov rax,%1
```

```
    mov rdi,%2
```

```
    mov rsi,%3
```

```
    mov rdx,%4
```

```
    syscall
```

```
%endmacro
```

```
section .data
```

```
menu db 10d,13d,"          MENU For Multiplication"
```

```
    db 10d,"1. Successive Addition"
```

```
    db 10d,"2. Shift and Add method"
```

```
    db 10d,"3. Exit"
```

```
    db 10d
```

```
    db 10d,"Enter your choice: "
```

```
Inmenu equ $-menu
```

```
    m1 db 10d,13d,"Enter First Number: "
```

```
    l1 equ $-m1
```

```
    m2 db 10d,13d,"Enter Second Number: "
```

```
    l2 equ $-m2
```

```
    m3 db 10d,13d,"Answer: "
```

```
    l3 equ $-m3
```

```
    nwline db 10d,13d
```

section .bss

choice resb 2

answer resb 20

num1 resb 20

num2 resb 20

temp resb 20

section .text

global _start

_start:

,*****MAIN LOGIC*****

main:

scall 1,1,menu,lnmenu

scall 0,0,choice,2

cmp byte[choice],'3'

jae exit

scall 1,1,m1,l1

scall 0,0,temp,17

call asciihextohex

mov qword[num1],rbx

scall 1,1,m2,l2

scall 0,0,temp,17

call asciihextohex

mov qword[num2],rbx

```
cmp byte[choice], '1'
```

```
je case1
```

```
cmp byte[choice], '2'
```

```
je case2
```

```
exit:
```

```
mov rax, 60
```

```
mov rdi, 0
```

```
syscall
```

```
case1:
```

```
mov rbx, qword[num1]
```

```
mov rcx, qword[num2]
```

```
mov rax, 0 ;rax to store answer
```

```
cmp rcx, 0 ;check multiplication with 0 condition
```

```
je skip3
```

```
loop1:
```

```
add rax, rbx
```

```
loop loop1 ;auto-decrement rcx and jmp
```

```
skip3:
```

```
mov rbx, rax ;backup rax in rbx
```

```
scall 1, 1, m3, l3
```

```
mov rax, rbx ;restore rax which has answer
```

```
call display ;display answer from rax register
```

```
jmp main
```

case2:

```
mov rbx,qword[num1]
mov rdx,qword[num2]
mov rax,0           ;rax to store answer
```

```
mov cl,64
```

; 16(no. of digit) * 4 (no. of bits per digit) = 64

up1:

```
shl rax,1
rol rbx,1
jnc down1
add rax,rdx
```

down1:

```
loop up1
mov rbx,rax
```

```
mov rbx,rax           ;backup rax in rbx
scall 1,1,m3,l3
mov rax,rbx           ;restore rax which has answer
call display          ;display answer from rax register
```

jmp main

*****PROCEDURES*****

asciihextohex:

```
mov rsi,temp
```

mov rcx,16

mov rbx,0

mov rax,0

loop4: rol rbx,04

mov al,[rsi]

cmp al,39h

jbe skip1

sub al,07h

skip1: sub al,30h

add rbx,rax

inc rsi

dec rcx

jnz loop4

ret

display:

mov rsi,answer+15

mov rcx,16

loop5: mov rdx,0

mov rbx,16

div rbx

cmp dl,09h

jbe skip2

```
    add dl,07h
skip2: add dl,30h
    mov [rsi],dl

    dec rsi
    dec rcx
    jnz loop5
    scall 1,1,answer,16
ret
```